Parent Monitoring System

📌 Project Overview

The Parent Monitoring System is a dual-component application designed to help parents monitor and manage their child's device usage. It consists of a Child Client application that runs on the child's device and a Parent Server dashboard that displays real-time activity, screen captures, and usage statistics. This system promotes digital safety and responsible device usage.

System Architecture

Two-Component Structure:

1. Child Client Application (child_client/)

- Runs on the child's device
- Captures screen activity
- Enforces usage restrictions
- Reports data to parent server

2. Parent Server Dashboard (Parent_server/)

- Web-based monitoring interface
- Real-time activity visualization
- Access control and settings management
- Historical data viewing

Project Structure

```
ParentMonitoringSystem/
|
├── child_client/              # Child-side application
|   ├── child_app.py           # Main child application
|   ├── monitor.py             # Activity monitoring module
|   ├── lock_screen.py         # Screen locking functionality
|   ├── config.py              # Child app configuration
|   ├── .env                   # Environment variables (child)
|   └── requirements.txt       # Dependencies for child app
|
├── Parent_server/             # Parent-side web server
|   ├── static/
|   |   ├── js/
|   |   |   └── dashboard.js   # Frontend JavaScript for dashboard
|   |   └── screenshots/       # Captured screen images storage
|   |
|   ├── templates/
|   |   ├── dashboard.html     # Main monitoring dashboard
|   |   └── login.html         # Parent authentication page
|   |
|   ├── app.py                 # Main Flask server
|   ├── config.py              # Server configuration
|   ├── .env                   # Server environment variables
|   └── requirements.txt       # Server dependencies
```

```
│
├── database/                # Database management
│   ├── schema.md            # Database schema documentation
│   └── seed_data.py         # Initial data population
│
├── docs/                    # Project documentation
│   ├── NGROK_GUIDE.md       # External access setup
│   └── SETUP.md             # Installation guide
│
├── venv/                    # Python virtual environment
├── C-B729D34E_20251230_110053.png  # Sample screenshot/asset
└── .gitkeep                 # Git placeholder file
```

Core Components

1. Child Client Application (child_client/)

Key Files:

- child_app.py – Main application loop and coordination
- monitor.py – Handles activity monitoring, screenshot capture, and usage tracking
- lock_screen.py – Implements screen locking functionality with configurable restrictions
- config.py – Child-side configuration (monitoring intervals, restrictions, server URL)

Features:

- Screen Capture: Periodic screenshot capture at configurable intervals
- Activity Monitoring: Tracks application usage and active window focus

- Usage Enforcement: Implements time limits and content restrictions
- Data Reporting: Sends collected data to parent server via API
- Screen Locking: Enforces "device timeout" when limits are reached

---

2. Parent Server (Parent_server/)

Key Files:

- app.py – Flask server with REST API endpoints and WebSocket support
- templates/dashboard.html – Main monitoring interface with real-time updates
- static/js/dashboard.js – Frontend logic with animations and live data fetching
- templates/login.html – Secure parent authentication portal

Features:

- Real-Time Dashboard: Live view of child's current activity
- Historical Data: Review past usage patterns and screenshots
- Settings Management: Configure restrictions and monitoring parameters
- Multi-Child Support: Monitor multiple devices/children
- Secure Access: Password-protected login system

🚀 Installation & Setup

Prerequisites:

- Python 3.8+
- pip package manager
- Windows/macOS/Linux (cross-platform compatible)

- Modern web browser for parent dashboard

Step-by-Step Setup:

1. Parent Server Setup:

```
# Navigate to parent server directory
cd ParentMonitoringSystem/Parent_server

# Create virtual environment
python -m venv venv

# Activate virtual environment
# Windows:
venv\Scripts\activate
# Linux/Mac:
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment variables
cp .env.example .env
# Edit .env with your settings (SECRET_KEY, DATABASE_URL, etc.)

# Initialize database
```

```
python -c "from app import db; db.create_all()"

# Run the server
python app.py
```

2. Child Client Setup:

```
bash
# Navigate to child client directory
cd ParentMonitoringSystem/child_client

# Create virtual environment
python -m venv venv

# Activate virtual environment
# Windows:
venv\Scripts\activate
# Linux/Mac:
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Configure environment variables
cp .env.example .env
# Edit .env with parent server URL and child device ID
```

```
# Run the child client
python child_app.py
```

Authentication & Security

Security Measures:

1. Parent Authentication:

- Secure login system for dashboard access
- Session management with timeout
- Password hashing using bcrypt/Werkzeug

2. Data Transmission:

- HTTPS/SSL recommended for production
- API token authentication between client and server
- Encrypted screenshot storage

3. Privacy Considerations:

- Local data storage options
- Clear data retention policies
- Parental consent compliance

Dashboard Interface

Main Dashboard Features:

- Live Activity Feed: Real-time updates of child's current activity

- Screenshot Gallery: Grid view of captured screens with timestamps
- Usage Statistics: Daily/weekly usage charts and patterns
- Restriction Controls: Toggle various restrictions and limits
- Device Status: Online/offline status and connection health

UI/UX Elements:

- Slide Animations: Smooth transitions for notifications and panels (as seen in dashboard.js)
- Responsive Design: Adapts to desktop and tablet screens
- Real-time Updates: WebSocket connections for instant data refresh
- Visual Indicators: Color-coded alerts for exceeded limits or issues

---

Configuration

Parent Server Configuration (.env):

env

```
SECRET_KEY=your_flask_secret_key
DATABASE_URL=sqlite:///parent_monitor.db
SERVER_HOST=0.0.0.0
SERVER_PORT=5000
DEBUG=False
SCREENSHOT_DIR=static/screenshots
DATA_RETENTION_DAYS=30
```

Child Client Configuration (.env):

env

```
PARENT_SERVER_URL=http://localhost:5000
DEVICE_ID=child_device_001
MONITOR_INTERVAL=60  # seconds between screenshots
ENABLE_SCREEN_LOCK=true
DAILY_LIMIT_MINUTES=120
ALLOWED_APPS=chrome,word,excel
```

📊 Database Schema

Key Tables:

1. Parents Table: Parent user credentials and preferences
2. Children Table: Child profiles and device information
3. Sessions Table: Active usage sessions with timestamps
4. Screenshots Table: Captured images with metadata
5. Activity Logs: Detailed application usage history
6. Restrictions: Custom rules and limits per child

(See database/schema.md for complete schema documentation)

API Endpoints

Parent Server API:

- POST /api/login – Parent authentication
- GET /api/dashboard-data – Dashboard statistics and recent activity
- POST /api/receive-activity – Child client data submission

- GET /api/screenshots/:child_id – Retrieve captured screenshots
- POST /api/update-restrictions – Modify child restrictions
- WS /websocket – Real-time WebSocket connection

Child Client Reporting:

- Periodic POST requests with activity data
- Screenshot uploads via multipart/form-data
- Heartbeat signals for connection monitoring

🛡️ Monitoring Features

What Gets Monitored:

1. Screen Activity: Periodic screenshots (configurable interval)
2. Application Usage: Active window tracking and time spent
3. System Metrics: Device uptime, network status
4. Content Awareness: Basic URL/application category detection
5. Attempt Logs: Restricted access attempts

Restriction Capabilities:

- Time Limits: Daily or session-based usage limits
- Application Blocking: Whitelist/blacklist for applications
- Schedule Enforcement: Allowed usage hours
- Content Filtering: Basic web/content filtering
- Screen Lock: Immediate lock when limits reached

🚨 Alerts & Notifications

Alert Types:

1. Limit Exceeded: Daily time limit reached
2. Restricted Access: Blocked application attempted
3. Device Offline: Child device disconnected
4. Anomaly Detection: Unusual usage patterns
5. Manual Alerts: Parent-initiated notifications

Notification Methods:

- Dashboard popups with slide animations
- Email alerts (if configured)
- Sound notifications
- Visual status indicators

🔧 Troubleshooting

Common Issues:

Issue	Solution
Child client not connecting	Verify PARENT_SERVER_URL in child .env file
Screenshots not appearing	Check folder permissions for static/screenshots/
Dashboard login fails	Ensure database is initialized and SECRET_KEY is set
High CPU usage on child device	Increase MONITOR_INTERVAL in child configuration
WebSocket connection drops	Check firewall settings and ngrok configuration

Debug Mode:

Enable debug mode in .env files for detailed logs:

env

DEBUG=True

LOG_LEVEL=DEBUG

Future Enhancements

Planned Features:

1. Mobile Applications: iOS/Android parent apps
2. Advanced Analytics: Machine learning for usage pattern detection
3. Geofencing: Location-based restrictions
4. Cross-Platform Support: Linux monitoring improvements
5. Cloud Sync: Optional cloud backup and multi-device sync
6. Reporting Tools: PDF reports for weekly/monthly reviews
7. API Expansion: Third-party integration capabilities

Technical Improvements:

- Docker containerization for easier deployment
- Performance optimization for low-resource devices
- Enhanced privacy controls and data encryption
- Automated testing suite
- Internationalization (multi-language support)

Ethical Considerations & Best Practices

Responsible Usage Guidelines:

1. Transparency: Inform children about monitoring
2. Consent: Appropriate for age and family values
3. Privacy Balance: Monitor only what's necessary
4. Data Security: Secure storage and transmission
5. Open Communication: Use as tool for discussion, not just control

Legal Compliance:

- COPPA (Children's Online Privacy Protection Act) considerations
- GDPR compliance for EU users
- Local privacy laws and regulations
- Clear data retention and deletion policies

Documentation Files

Included Guides:

- docs/NGROK_GUIDE.md – Setting up external access via ngrok
- docs/SETUP.md – Detailed installation instructions
- database/schema.md – Complete database documentation

Contributing

1. Fork the repository
2. Create a feature branch (git checkout -b feature/improvement)
3. Commit changes (git commit -m 'Add new feature')
4. Push to branch (git push origin feature/improvement)
5. Open a Pull Request